UNIT III – Constraint Satisfaction Problems & Game Theory

AI & ML for Mechanical Engineering (23A03701)

➤ Syllabus of this unit

- Local search for constraint satisfaction problems (CSP)
- Adversarial search, games, optimal decisions & strategies in games
- The minimax search procedure, alpha-beta pruning
- Additional refinements, iterative deepening

1. Constraint Satisfaction Problems (CSP)

➤ Why CSP?

In ordinary search a state is a ***black box*** – we only know "goal or not goal". In a CSP the state has an ***internal structure*** (variables with values), so we can reject bad partial assignments early. This makes CSP solvers far faster than blind search.

➤ Definition

A CSP is defined by three components **(X, D, C)**:

- **X** = set of variables {X1, X2, ..., Xn}
- **D** = set of domains {D1, ..., Dn}; each Di is the set of allowed values for Xi
- **C** = set of constraints that specify allowed combinations of values

- ***Assignment*** – giving values to some / all variables.
- ***Consistent (legal) assignment*** – violates no constraint.
- ***Complete assignment*** – every variable has a value.
- ***Solution*** = a complete and consistent assignment.
- ***Partial assignment*** – only some variables assigned.

➤ Types of constraints

Type	Meaning	Example
Unary	restricts a single variable	SA ≠ green
Binary	relates two variables	SA ≠ WA
Global (n-ary)	involves many variables	Alldiff(X1..X9) in Sudoku
Preference (soft)	desirable, not compulsory	prefer red to green (cost)

➤ Types of variables / domains

- **Discrete finite** domain – map colouring, N-queens, scheduling.
- **Discrete infinite** – integers, strings (needs constraint language).
- **Continuous** – start/end times of jobs; solved by linear programming.

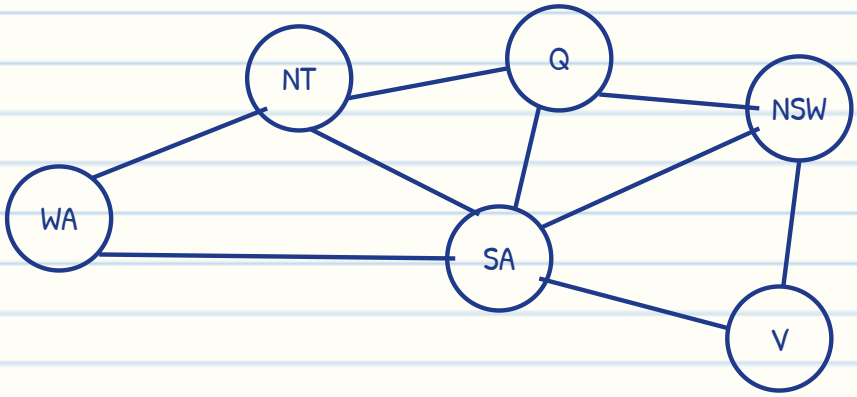
➤ Mechanical engineering applications

- Job-shop / machine scheduling (no two jobs on one machine together)
- Assembly sequencing of parts in a gearbox
- Timetabling, cutting-stock, pipe-routing, layout of components

2. Standard CSP Examples

➤ (a) Map colouring – Australia

Variables: WA, NT, Q, NSW, V, SA, T. Domain: {red, green, blue}. Constraint: adjacent regions get different colours, e.g. WA ≠ NT, WA ≠ SA, NT ≠ SA, NT ≠ Q, SA ≠ Q, SA ≠ NSW, SA ≠ V, Q ≠ NSW, NSW ≠ V.



T (Tasmania) – isolated, no edges

Constraint graph: node = variable, edge = binary constraint

> **One solution:** WA=red, NT=green, SA=blue, Q=red, NSW=green, V=red, T=any colour. T is an independent sub-problem – it can be solved separately.

➤ (b) N-Queens (4-queens)

Place N queens on N×N board so no two attack (same row, column, diagonal). **Formulation:** variable Qi = row of the queen in column i; domain {1..N}; constraints Qi ≠ Qj and |Qi – Qj| ≠ |i – j|.

➤ 4-queens solution

(Q1,Q2,Q3,Q4) = (2, 4, 1, 3) and its mirror (3, 1, 4, 2). Only 2 solutions exist for N = 4.

➤ (c) Cryptarithmetic

SEND + MORE = MONEY. Each letter a different digit. Constraints: Alldiff, column sums with carries C1..C3. Solution: S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2 (9567+1085=10652).

➤ (d) Others

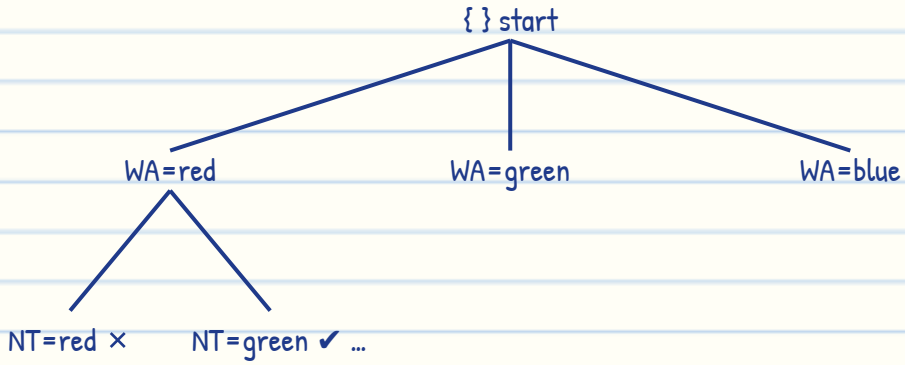
Sudoku (81 variables, Alldiff on rows, columns, boxes), Boolean satisfiability, scheduling of machining operations.

3. Solving CSPs – Backtracking Search

CSP assignments are ***commutative***: order of assignment doesn't matter ([WA=red then NT=green] same as reverse). So at each node we assign only one variable – branching factor stays d, not nd. Total leaves d^n, not $n! \cdot d^n$.

➤ Backtracking algorithm (DFS for CSP)

```
function BACKTRACK(assignment, csp):
  if assignment is complete: return assignment
  var = SELECT-UNASSIGNED-VARIABLE(csp)
  for each value in ORDER-DOMAIN-VALUES(var):
     if value consistent with assignment:
        add {var = value} to assignment
        result = BACKTRACK(assignment, csp)
        if result != failure: return result
        remove {var = value}      # undo
  return failure
```



Backtracking tree: NT=red conflicts with WA=red so we back up

Plain backtracking is still slow. Three questions improve it:

1. Which variable next? (variable ordering)
2. Which value first? (value ordering)
3. Can we detect failure early? (inference)

4. Heuristics for Backtracking

➤ Variable ordering

- **MRV – Minimum Remaining Values** (fail-first / most constrained variable): choose the variable with the fewest legal values left. E.g. after WA=red, NT=green, SA has only {blue}, so pick SA next. Exposes failure early.
- **Degree heuristic** (tie-breaker): choose the variable involved in the most constraints on unassigned variables. At the start SA (degree 5) is chosen first.

➤ Value ordering

- **LCV – Least Constraining Value**: choose the value that rules out the fewest choices for neighbouring variables, leaving maximum flexibility.

> **Remember:** for variables use "fail-first" (MRV); for values use "fail-last" (LCV) – because we need just one solution.

5. Inference – Constraint Propagation

Constraint propagation uses constraints to **reduce domains** of variables, which in turn reduces others – done as pre-processing or during search.

➤ Forward Checking (FC)

When X is assigned, delete from each unassigned neighbour Y's domain any value inconsistent with X. If any domain becomes empty → backtrack immediately.

Step	WA	NT	Q	NSW	V	SA
Initial	RGB	RGB	RGB	RGB	RGB	RGB
WA = red	R	GB	RGB	RGB	RGB	GB
Q = green	R	B	G	RB	RGB	B
V = blue	R	B	G	R	B	∅

SA's domain is empty after V=blue, so FC detects failure **before** trying more. But note NT and SA are both only {blue} and adjacent – FC doesn't notice this! We need stronger propagation.

➤ Arc Consistency (AC-3)

➤ Definition

Arc X → Y is **consistent** if for every value x in D(X) there is some value y in D(Y) satisfying the constraint between X and Y.

If a value x has no support in D(Y), delete x from D(X). Repeat; when D(X) shrinks, re-check all arcs (Z → X) pointing into X.

```
function AC-3(csp):
  queue = all arcs in csp
  while queue not empty:
    (Xi, Xj) = REMOVE-FIRST(queue)
    if REVISE(csp, Xi, Xj):
      if D(Xi) is empty: return false
      for each Xk in neighbours(Xi) - {Xj}:
        add (Xk, Xi) to queue
  return true
```

Example: X, Y ∈ {0..9}, constraint $Y = X^2$. Arc X → Y: X can only be {0,1,2,3}; arc Y → X: Y only {0,1,4,9}.
Complexity of AC-3: $O(c \cdot d^3)$ (c = no. of arcs).

- ***Node consistency*** – unary constraints satisfied.
- ***Path consistency*** – any consistent pair of 2 variables extends to a third.
- ***k-consistency*** – generalisation; strong n-consistency solves the CSP without backtracking.

MAC = Maintaining Arc Consistency: run AC-3 after every assignment during backtracking.

➤ Intelligent backtracking

- ***Chronological*** backtracking goes to the most recent variable.
- ***Backjumping*** jumps to the most recent variable in the *conflict set* – the actual cause of failure.
- ***Conflict-directed backjumping*** and ***constraint learning*** (store no-goods) refine this further.

➤ Problem structure

- Independent subproblems (Tasmania) – connected components of the constraint graph solved separately: reduces d^n to $(n/c) \cdot d^c$.
- ***Tree-structured CSP*** solved in $O(n \cdot d^2)$: pick root, order variables topologically, make arcs consistent from leaves up, assign from root down.
- ***Cutset conditioning*** – assign a cutset of variables so remaining graph becomes a tree.

6. Local Search for CSPs

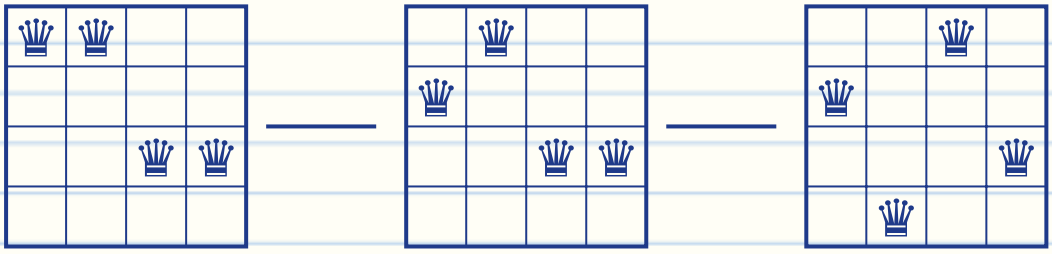
Backtracking builds a solution step by step. ***Local search*** starts with a complete assignment (usually with conflicts) and repairs it by changing one variable at a time until all constraints are satisfied. Memory needed is tiny – excellent for huge problems (e.g. million-queens solved in ~50 steps).

➤ Min-Conflicts heuristic

```
function MIN-CONFLICTS(csp, max_steps):
  current = a complete (random) assignment
  for i = 1 to max_steps:
    if current is a solution: return current
    var = randomly chosen CONFLICTED variable
    value = value of var that MINIMISES number of conflicts
    set var = value in current
  return failure
```

➤ 4-Queens by min-conflicts

Start: columns have queens in rows (1,1,3,3) → several attacks (h = conflicts). Pick a conflicted queen, move it vertically within its column to the row with fewest attacks. Typically 1–3 moves reach a solution such as (2,4,1,3).



h = 5 h = 2 h = 0 ✔

Min-conflicts reduces conflicts (h) at each step

- Landscape may have ***plateaus*** – use ***tabu search*** (remember recent moves) or ***random walk***.
- Can use ***constraint weighting***: increase weight of constraints that stay violated so search is pushed out of local minima.
- Also hill-climbing / simulated annealing (Unit II) work on CSPs with h = number of violated constraints.
- Advantage: handles ***online*** changes (re-scheduling after a machine breakdown) by repairing the current schedule.

Feature	Backtracking	Local search (min-conflicts)
State	partial assignment	complete assignment
Complete?	Yes	No (may get stuck)
Memory	depth n	O(1) – only current state
Best for	small, tight problems	large, loosely constrained

7. Adversarial Search & Games

In ***multi-agent competitive*** environments, the goals of agents conflict – e.g. chess, tic-tac-toe, checkers. The opponent's moves are unpredictable, so we need a ***strategy*** (what to do for every possible reply), not just a fixed path. Game playing was one of the first AI tasks (Shannon 1950, Turing).

Why games are hard: huge search space (chess: branching ≈ 35, depth ≈ 100 → 35^{100} nodes), and there is a time limit per move – so we must make the *best possible decision* with imperfect search.

➤ Types of games

	Deterministic	Chance (stochastic)
Perfect information	Chess, Checkers, Go, Tic-tac-toe	Backgammon, Monopoly
Imperfect information	Battleship	Bridge, Poker, Scrabble

We study ***two-player, zero-sum, deterministic, perfect-information, turn-taking*** games. Zero-sum: one player's gain = other's loss.

➤ Formal definition of a game

- ***S0*** – initial state
- ***PLAYER(s)*** – whose turn in state s (MAX or MIN)
- ***ACTIONS(s)*** – legal moves
- ***RESULT(s,a)*** – transition model
- ***TERMINAL-TEST(s)*** – is game over?
- ***UTILITY(s,p)*** – final payoff (chess: +1 win, 0 draw, −1 loss)

Two players: ***MAX*** (tries to maximise utility, moves first) and ***MIN*** (tries to minimise). The ***game tree*** has nodes = states, edges = moves; a level of one player's move is a ***ply***.

➤ Optimal decisions & strategies

In normal search the solution is a sequence of actions. In a game MAX must find a contingent strategy: a move for the initial state, then moves for every possible reply of MIN, and so on. An ***optimal strategy*** gives outcome at least as good as any other strategy *when playing against an infallible opponent.*

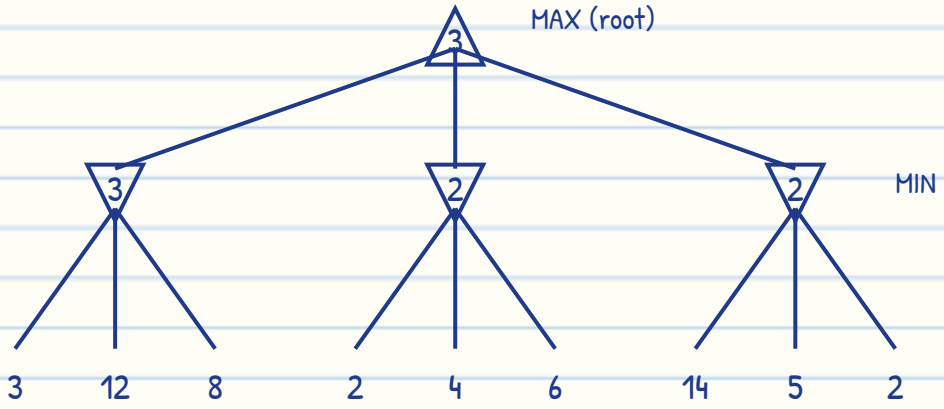
8. The Minimax Procedure

➤ Minimax value

```
MINIMAX(s) =
  UTILITY(s)                    if TERMINAL-TEST(s)
  max over a of MINIMAX(RESULT(s,a))  if PLAYER(s) = MAX
  min over a of MINIMAX(RESULT(s,a))  if PLAYER(s) = MIN
```

Idea: choose the move leading to the position with the highest ***minimax value*** – the best achievable utility assuming MIN plays optimally. Values are computed by DFS and ***backed up*** from the leaves.

```
function MINIMAX-DECISION(state):
  return argmax over a in ACTIONS(state) of MIN-VALUE(RESULT(state,a))
function MAX-VALUE(state):
  if TERMINAL-TEST(state): return UTILITY(state)
  v = -infinity
  for each a: v = MAX(v, MIN-VALUE(RESULT(state,a)))
  return v
function MIN-VALUE(state):
  if TERMINAL-TEST(state): return UTILITY(state)
  v = +infinity
  for each a: v = MIN(v, MAX-VALUE(RESULT(state,a)))
  return v
```



leaf utilities: MIN picks min of children: 3, 2, 2 → MAX picks max = 3 (move a1)

Worked minimax tree (the classic example)

Working: B = min(3,12,8) = 3; C = min(2,4,6) = 2; D = min(14,5,2) = 2. Root A = max(3,2,2) = **3**. So MAX plays a1 (towards B).

➤ Properties of minimax

Property	Value
Complete?	Yes, if the tree is finite
Optimal?	Yes, against an optimal opponent
Time	$O(b^m)$
Space	O(bm) (DFS, all actions at once) or O(m) (one at a time)

b = legal moves per position, m = max depth. For chess b≈35, m≈100 → completely infeasible, hence ***pruning***.

➤ Multi-player games

Replace single value by a ***vector*** of utilities, one per player; each player maximises own component. Alliances may form implicitly.

9. Alpha-Beta Pruning

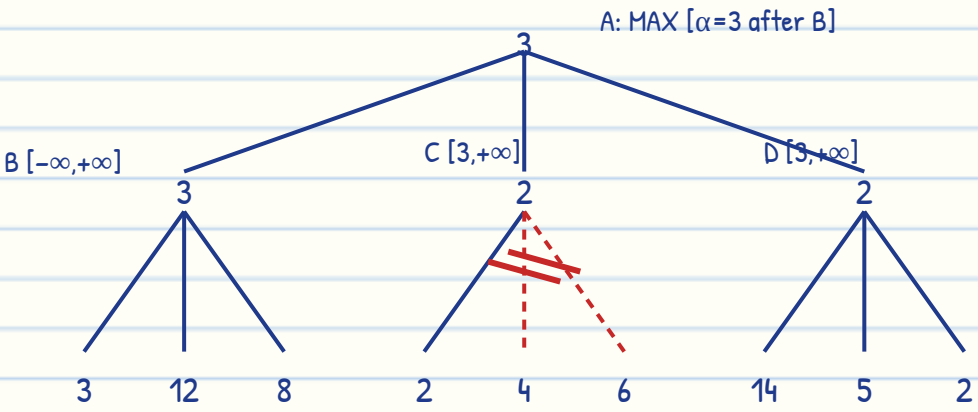
Minimax examines a number of nodes ***exponential*** in depth. Alpha-beta pruning returns the same minimax decision but ignores branches that cannot influence the final decision. It can cut the effective depth searched in half.

➤ Definitions

- α ***(alpha)*** = value of the ***best (highest)*** choice found so far for MAX along the path. Initially $-\infty$.
- β ***(beta)*** = value of the ***best (lowest)*** choice found so far for MIN along the path. Initially $+\infty$.
- ***Prune*** (stop exploring the remaining children) whenever $\alpha \geq \beta$.

```
function ALPHA-BETA-SEARCH(state):
  v = MAX-VALUE(state, -inf, +inf); return action with value v
function MAX-VALUE(state, alpha, beta):
  if TERMINAL-TEST(state): return UTILITY(state)
  v = -inf
  for each a in ACTIONS(state):
    v = MAX(v, MIN-VALUE(RESULT(state,a), alpha, beta))
    if v >= beta: return v        # beta cut-off
    alpha = MAX(alpha, v)
  return v
function MIN-VALUE(state, alpha, beta):
  if TERMINAL-TEST(state): return UTILITY(state)
  v = +inf
  for each a in ACTIONS(state):
    v = MIN(v, MAX-VALUE(RESULT(state,a), alpha, beta))
    if v <= alpha: return v       # alpha cut-off
    beta = MIN(beta, v)
  return v
```

➤ Worked example (same tree as minimax)



pruned! 4, 6 never examined

leaves of D: 14 → β=14; 5 → β=5; 2 → β=2 (all seen, no cut because 2 < 3 only at last)

Dashed red branches are pruned

Trace:

1. Visit B ($\alpha=-\infty, \beta=+\infty$): leaves 3, 12, 8 → B = 3. Back at A: α = 3.
2. Visit C with [$\alpha=3, \beta=+\infty$]: first leaf = 2 → v = 2 ≤ $\alpha(3)$ ⇒ ***cut-off***. Leaves 4 and 6 are skipped. C ≤ 2, so MAX will never choose C.
3. Visit D [3, +∞]: leaves 14 ($\beta=14$), 5 ($\beta=5$), 2 (v=2 ≤ 3 → value 2). D = 2.
4. Root = max(3, 2, 2) = 3. Same answer as minimax with 2 fewer leaf evaluations.

➤ Effectiveness & move ordering

- Pruning does not change the result – only efficiency.
- ***Worst case*** (bad ordering): no pruning, $O(b^m)$.
- ***Best case*** (perfect ordering, best moves first): $O(b^{m/2})$ – can search twice as deep in the same time. Chess: effective branching drops from 35 to about 6.
- ***Random ordering***: roughly $O(b^{3m/4})$.
- ***Move ordering heuristics:*** try captures first, then threats, forward moves, backward moves (chess); use ***killer moves*** (moves that caused cut-offs earlier at same depth); use ***iterative deepening*** results to order moves.
- ***Transposition table*** – hash table storing evaluated positions to avoid recomputing repeated states reached by different move orders.

Minimax	Alpha-beta
Explores whole tree to depth limit	Prunes irrelevant branches
$O(b^m)$	$O(b^{m/2})$ best case
Needs no α, β	Maintains α, β bounds
Same final decision	Same final decision

10. Additional Refinements

➤ (a) Imperfect real-time decisions – cutoff test

Searching to terminal states is impractical, so (Shannon 1950) we replace:

- TERMINAL-TEST → ***CUTOFF-TEST*** (e.g. depth limit d, or time up)
- UTILITY → ***EVAL*** (heuristic evaluation function estimating utility)

H-MINIMAX(s,d) = EVAL(s) if CUTOFF-TEST(s,d); else max/min of H-MINIMAX over successors with d+1.

➤ (b) Evaluation functions

Should (1) agree with true utility on terminal states, (2) be fast, (3) correlate with the real chance of winning.

Typically a ***weighted linear function***:

$EVAL(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$

Chess: f = (no. of white pieces − black pieces) of each type; weights pawn=1, knight/bishop=3, rook=5, queen=9; extra features: king safety, pawn structure, mobility. Weights can be ***learned*** from experience.

Tic-tac-toe example: EVAL = (rows/cols/diagonals still open for MAX) − (those open for MIN).

➤ (c) Quiescence search

EVAL is reliable only in ***quiescent*** positions – ones unlikely to change wildly soon (e.g. no pending capture). At the cutoff depth, if the position is non-quiescent, extend the search until it becomes quiet.

➤ (d) Horizon effect

A damaging move by the opponent is unavoidable, but the program delays it with delaying tactics (useless checks, sacrifices) so that it falls ***beyond the search horizon*** – and wrongly believes the problem is gone. Remedies: ***singular extensions*** (extend search for a clearly-best move), quiescence search, deeper search.

➤ (e) Forward pruning

- ***Beam search*** – consider only the n best moves at each ply (risky: best move may be pruned).
- ***ProbCut*** – prune moves statistically likely to be outside the (α, β) window.
- Late-move reductions: search later moves in the list to lesser depth.

➤ (f) Table look-up

Use ***opening books*** (first few moves from human experience) and ***endgame databases*** (perfect play computed backwards for few pieces left).

➤ (g) Stochastic games (chance)

Games with dice (backgammon): add ***chance nodes*** between MAX and MIN levels. Use ***expectiminimax***:

```
EXPECTIMINIMAX(s) =
  UTILITY(s)                          if terminal
  max over a  EXPECTIMINIMAX(RESULT(s,a)) if MAX
  min over a  EXPECTIMINIMAX(RESULT(s,a)) if MIN
  sum over r  P(r) · EXPECTIMINIMAX(RESULT(s,r)) if CHANCE
```

Coin-flip chance node (P=0.5 each) over MIN values 2 and 4: expected value = 0.5×2 + 0.5×4 = **3**. Complexity $O(b^m n^m)$, n = number of dice rolls – deep look-ahead is hard. Evaluation functions must be positive linear transforms of win probability. Monte-Carlo simulation (rollouts) is an alternative; Monte-Carlo tree search (MCTS) drives AlphaGo.

➤ (h) Partially observable games

Games like Bridge, Poker involve hidden information – handled by averaging over possible hidden states (card-deal sampling) or belief-state reasoning.

11. Iterative Deepening in Games

Because the time per move is fixed (e.g. 2 minutes), we do not know in advance what depth will finish. ***Iterative deepening search (IDS)***: run depth-limited alpha-beta with depth 1, then 2, then 3 ... until time runs out, and return the move from the deepest completed iteration.

```
function ITERATIVE-DEEPENING(state, time_limit):
  best = none
  for depth = 1, 2, 3, ...:
    if time is up: break
    best = ALPHA-BETA-SEARCH(state, depth)
  return best
```

Advantage	Explanation
Anytime algorithm	always has a move ready when the clock expires
Little overhead	repeated shallow levels cost small fraction because most nodes are at the bottom level (≈ b/(b−1) times extra)
Better move ordering	best move of depth d−1 is tried first at depth d → more alpha-beta cut-offs
Memory-efficient	DFS-like space O(bd)

Combined with ***transposition tables*** (store best move per position), results of earlier iterations are reused.

12. Quick Summary

- CSP = (X, D, C); solution is complete + consistent assignment.
- Backtracking + MRV + degree + LCV + forward checking / AC-3 (MAC) solves CSPs efficiently.
- Local search (min-conflicts) repairs a complete assignment – great for large CSPs.
- Minimax gives optimal play against an optimal opponent; alpha-beta prunes (prune when $\alpha \geq \beta$), best case $O(b^{m/2})$.
- Real games need cutoff + EVAL, quiescence, horizon-effect handling, chance nodes (expectiminimax), iterative deepening.

13. Important Exam Questions & Short Answers

Q1. Define CSP. Give an example.

A problem with variables X, domains D and constraints C; solution is a complete consistent assignment. Example: map colouring of Australia with 3 colours, adjacent regions different.

Q2. Differentiate forward checking and arc consistency.

FC only checks neighbours of the *just-assigned* variable and deletes inconsistent values; AC-3 propagates repeatedly through all arcs, so it detects failures FC misses (e.g. adjacent NT, SA both {blue}).

Q3. What are the MRV, degree and LCV heuristics?

MRV: variable with fewest legal values. Degree: variable with most constraints on unassigned variables (tie-break). LCV: value that rules out fewest values in neighbours.

Q4. Why is min-conflicts effective?

It repairs a complete assignment by moving a conflicted variable to its least-conflict value; the solution density is high in many problems, so it needs few steps and O(1) extra memory. Fails on plateaus unless tabu / random walk is used.

Q5. What is a zero-sum game? What are MAX and MIN?

A game where total payoff is constant (one's gain is other's loss). MAX maximises, MIN minimises the utility of the terminal state.

Q6. Write minimax and its complexity.

Recursive DFS; value of a MAX node = max of children, MIN node = min of children. Time $O(b^m)$, space $O(bm)$.

Q7. Explain alpha-beta pruning with an example.

Maintain α (best for MAX) and β (best for MIN); stop expanding a node when $\alpha \geq \beta$. In the example tree, leaves 4 and 6 under C are pruned after seeing 2 because $2 \leq \alpha = 3$. Gives same result as minimax; best case $O(b^{m/2})$.

Q8. What is the horizon effect and how is it reduced?

Delaying an inevitable loss beyond the search depth so it looks avoided. Reduced by quiescence search and singular extensions.

Q9. Why iterative deepening in game playing?

Time limited; it gives an anytime answer, small overhead, and improves move ordering so alpha-beta prunes more.

Q10. What is expectiminimax?

Extension of minimax for stochastic games with chance nodes whose value is the probability-weighted average of children.

~ End of Unit III ~ All the best! ~